Nil Seri
Apr 24 · 7 min read · Listen

Microservices — Transaction Management Patterns

Some Popular Microservices Transaction Management Patterns and Distributed System Problems

Photo by Denise Johnson on Unsplash

· · ·

Write-ahead logging (WAL): provides atomicity and durability (**ACID**) in database systems. The chang 83 ··· the log, which must be written to stable storage, before the changes are written to the database.

- **Undo log:** allows a dbms to undo a transaction. a version of modified data is saved before a transaction starts. The generation of undo logs is accompanied by the generation of redo logs.
- **Redo log:** allows a dbms to redo a transaction. consists of files that store all changes made to the database as they occur. Every instance of an Oracle Database has an associated redo log to protect the database in case of an instance failure.

· · ·

Two Phase Commit (2PC) ❌

Assumptions:
- One process is "coordinator" (C), the rest are "cohort"'s (H).
- Coordinator can be initiator (or not).
- Storage is stable, no machine crashes.
- Write-ahead log exists at each site.

Phase 1 (Voting Phase):
- C sends a "query" msg to all H's and wait for replies.
- All H's work on txn until commit phase. They write an entry to their *undo* log and an entry to their *redo* log. Each H sends "agree" or "abort" reply depending on their trxn's outcome.

Phase 2 (Completion Phase):
Success:
- C sends a "commit" msg to all H's.
- H's complete operation, release all the locks and resources held during the transaction.
- H's send "acknowledgement" msg to C. C completes trxn when all "acknowledgement" msgs are received.
Failure:
- C sends a "rollback" msg to all H's.
- H's undo trxn using the *undo* log, release all the locks and resources held during the transaction.
- H's send "acknowledgement" msg to C. C undoes trxn when all "acknowledgement" msgs are received.

```
- not appropriate for microservices ❌
- if C fails, it's a single point of failure. "Availability" impacted.
- depends on slowest response time, locks everywhere
- not supported by NoSQL dbs & msg brokers
```

Saga Pattern ✔

- Single independent unit of work
- Every microservice should expose 1. participate 2. rollback (compensation) methods. (for n transaction -> n compensation)
- If one in the chain fails, all the previously occurred ones' rollbacks should be called.
- Compensation service must be idempotent (cannot abort) and must be retried until success.

1. **Orchestration Based:**
One coordinator. Single point of failure problem still exists.
2. **Choreography Based:**
Each local trxn publishes domain events that trigger local trxns in other services. Preferred over orchestration-based.

```
- SAGA -> ACD ✔ I ❌ : does not guarantee Isolation
  Countermeasures:
  * Versioning data (check version before update)
  * Re-read data (before modifying)
  * Semantic locking (lock flag preventing other txns from accessing
it)
```

· · ·

CQRS (Command and Query Responsibility Segregation) Pattern

- Queries (read)
- Commands (update)

CQRS separates read and update operations to maximize performance. It is suitable especially for systems where reads >> writes and where stale data is no problem.

Separated databases -> C❌ (AP✔) Theorem

CQRS Implementation — https://betterprogramming.pub/cqrs-software-architecture-pattern-the-good-the-bad-and-the-ugly-e9d6e7a34daf

```
- complex
- database dependent, requires expertise in a variety of database
technologies.
- using a large number of databases means more points of failure.
- need to handle data consistency in terms of CAP
```

Event Sourcing

- All changes to app state are stored on a sequence of events (journal of domain events) (easy to roll back). (e.g. Axon)
- Commonly combined with the CQRS pattern
- Defines an approach where all the changes that are made to an object or entity (also known as the write model), are stored as a sequence of immutable events to an event store, as opposed to storing just the current state.

```
- suitable for event-driven architecture
- makes it possible to reliably publish events whenever state changes
- audit log, easy to get state at a given time
- difficult to query event store (inefficient complex queries)
```

· · ·

Transactional Outbox Pattern

microservices -> database transaction + publish events => should be atomic.
What happens in case of system failure? => *Solution:* outbox table (since this is also a database table)
There may be a scanning batch job to check if any such record exists (e.g column "status=PROCESSED"), read from there and then publish related event.

Change Data Capture (CDC)

- It captures change events from a database log (or equivalent) (allowing application state to be externalized), forwards to consumers (e.g. Debezium).
- Types: Change Columns (such as DATE_MODIFIED) (not good at capturing deletes), Trigger-Based (performance issues), Log-Based (most preferred)

Log-Based Change Data Capture

- Directly integrated with database
MySQL, Postgres, etc. -> transaction log (write-ahead log) -> Debezium -> Kafka Queues

· · ·

Domain events — An explicit event, part of your business domain, that is generated by your application. These events are usually represented in the past tense, such as OrderPlaced, or ItemShipped. These events are the primary concern for *Event Sourcing*.

Change events — Events that are generated from a database transaction log indicating what state transition has occurred. These events are of concern for *Change Data Capture*.

Event Sourcing vs. Change Data Capture: Event Sourcing uses its own journal as the resource of reality, while Change Data Capture depends on the underlying database transaction log as the source of truth.

Comparison of Attributes — https://debezium.io/blog/2020/02/10/event-sourcing-vs-cdc/

· · ·

Listen to Yourself Pattern

- Consume the messages you yourself produced (db save comes later). => If the message was published to the message broker, you assume a consistent state.
- All your events and database writes must be idempotent (as in every microservice regardless of the pattern you are using).

Listen to Yourself Pattern — https://medium.com/@odedia/listen-to-yourself-design-pattern-for-event-driven-microservices-16f97c3ed066

```
- The risk of consistency is more since db write occurs later.
```

· · ·

Dead Letter Queue Pattern

- Data (in the form of a message from a queue) cannot be processed due to:
- the lack of access to some component part of the system
- the input data is invalid or corrupt.
- Solution => move the message to a "*dead-letter* queue"
design your system in a way that it not only limits the number of *re-processing* attempts, but also *postpones* the re-processing of failed messages (*parking-lot* queue) so as to allow other (potentially not accessible) parts of the system to become available.

Parking Lot and Dead Letter Queues — https://www.ibm.com/cloud/architecture/architectures/event-driven-deadletter-queue-pattern/

· · ·

Distributed System Models

Capture assumptions in a system model consisting of:

- Network behaviour (e.g. message loss)
- Node behaviour (e.g. crashes)
- Timing behaviour (e.g. latency)

The Two Generals Problem

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

- Messages may get lost.
- Both parts cannot know if their msgs are received without receiving an answer from the other. => This results in an infinite chain of msgs to check certainty.

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

As a microservice example:

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

- If payment succeeds but there are not enough goods in the shop => you can send an apology e-mail and refund card.
- There are enough goods in the shop but the shop did not get any information about the payment's outcome => ask payment service if it did actually charge that card.
- With the steps above, online shopping avoids having two generals problem.

· · ·

The Byzantine Generals Problem

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

Assumptions:
1. Messaging is reliable (messengers do not get captured and msgs get received).
2. Some of the generals are not reliable and may work together in this.

Aim: We want honest generals to succeed.

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

- Up to f generals might behave maliciously.
- Honest generals don't know who the malicious ones are.
- The malicious generals may collude.
- Nevertheless, honest generals must agree on plan.

Theorem: need 3f+1 generals in total to tolerate f malicious generals (i.e. < 1/3 may be malicious). Cryptography (digital signatures) helps but problem remains hard.

As a microservice example:

https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf

· · ·

Happy Coding!

· · ·

> ***References:***

https://dzone.com/articles/transaction-management-in-microservice-architectur
https://en.wikipedia.org/wiki/Two-phase_commit_protocol
https://betterprogramming.pub/cqrs-software-architecture-pattern-the-good-the-bad-and-the-ugly-e9d6e7a34daf
https://www.redhat.com/architect/pros-and-cons-cqrs
https://www.linkedin.com/feed/update/urn:li:activity:6923600979716124672
https://microservices.io/patterns/data/event-sourcing.html
https://debezium.io/blog/2020/02/10/event-sourcing-vs-cdc/
https://medium.com/mercedes-benz-techinnovation-blog/change-data-capture-lessons-learnt-7976391cf78d
https://en.wikipedia.org/wiki/Write-ahead_logging
https://docs.oracle.com/cd/E18283_01/server.112/e17120/onlineredo001.htm
https://www.alibabacloud.com/blog/what-are-the-differences-and-functions-of-the-redo-log-undo-log-and-binlog-in-mysql_598035
https://blog.cykerway.com/posts/2018/11/18/database-undo-log-and-redo-log.html
https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf
https://medium.com/@odedia/listen-to-yourself-design-pattern-for-event-driven-microservices-16f97c3ed066
https://medium.com/eleveo-techblog/dead-letter-queue-reprocessing-a2c041f64e65

83

More from Nil Seri

Follow

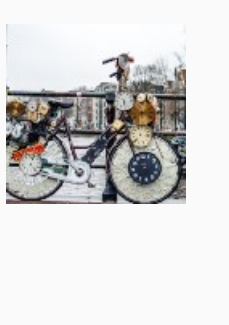
I would love to change the world, but they won't give me the source code | coding | coffee | jazz | anime | books | drawing

Feb 14

Object Oriented Design Principles Cheat Sheet

Concepts and Lists of Principles TL;DR My aim here is not to go into detail but create a cheatsheet to quickly review before interviews to serve as a reminder. OOP Concepts Abstraction (hide complexity — blackbox, show…

Object Oriented Design · 2 min read



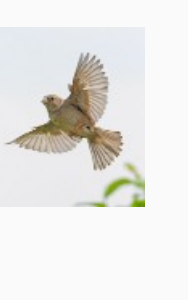
Share your ideas with millions of readers. Write on Medium

Feb 13

REST API Naming Standards & Best Practices

A Summary of REST API Naming Standards and Best Practices HTTP Request Methods POST — Create like form submit, to send data to server. (PUT is idempotent, POST is not) GET — Read (No Request Body) PUT …

Rest Api · 3 min read

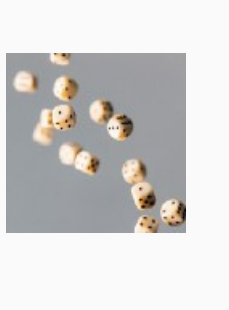


Feb 12

System Design Interview — Study Notes I

Notes on Concepts & Components to be used in a System Design Interview TL;DR These are my notes (and references for this post) from Udemy's "Mastering the System Design Interview" and Educative's "Grokking the…

System Design Interview · 12 min read

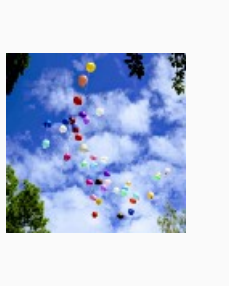


Jan 25

Unit Testing Concepts & Its Implementation in Java with PowerMock (and Mockito)

Unit Testing Concepts and Principles, Unit Testing in Java using PowerMock and Mockito Unit Testing Unit testing validates the function of a unit,…

Unit Testing · 5 min read

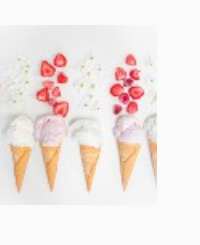


Jan 24

Parallel and Concurrent Programming (Using Java) — Part II

Notes on Parallel and Concurrent Programming To refresh my knowledge about concurrency and parallel programming, I took "Parallel and…

Interview Preparation · 9 min read



Love podcasts or audiobooks? Learn on the go with our new app. Try Knowable

Get unlimited access

Search

Nil Seri
571 Followers

I would love to change the world, but they won't give me the source code | coding | coffee | jazz | anime | books | drawing

Follow

More from Medium

Mohammed Ragab in Nerd For Tech
Eventual consistency consequences in the event-driven system

Sudheer Sandu
Building Reliable Systems

Hana Almadi
When do we need to implement event sourcing pattern ?

Tegar Budi Sep... in Blibli.com Tech ...
Spring Cloud Gateway and Its Resilience